Atelier 1 : Git et GitHub

Notions abordées :

- **Dépôt local : zones de travail, commits, historique.**
- **Annulation des changements.**
- **Branches, fusion et résolution de conflits.**
- **Dépôt distant GitHub : push, clone, pull.**

I. Mise en place d'un nouveau dépôt Git

Pour la mise en place d'un dépôt Git, deux options sont possibles :

1. Création d'un nouveau dépôt local, c.-à-d. :
   - création d'un dépôt pour un répertoire vide (pour commencer un nouveau projet) ;
   - création d'un dépôt pour un projet existant sur votre machine locale.
2. Clonage d'un dépôt existant sur GitHub, pour travailler sur cette copie de travail.

Nous commencerons par examiner la première option. *Nous nous servirons de la ligne de commande.*

La commande **`git init`** permet d'initialiser un dépôt Git dans le répertoire en question. Dès lors, ce dernier contiendra à la fois une version de travail (dans le répertoire) et un dépôt Git (dans `TpDepot/.git`).

1. **Configuration :** tâche administrative, à effectuer une seule fois. Configurez le nom d'utilisateur et l'email de Git, ainsi que le nom de la branche créée par défaut.

   définir votre nom : **`git config --global user.name "votre nom"`**
   définir votre email : **`git config --global user.email "votre adresse email"`**
   nommer la branche par défaut : **`git config --global init.defaultBranch main`**

   La commande **`git config --list`** permet de vérifier que le nom et l'email ont été définis.

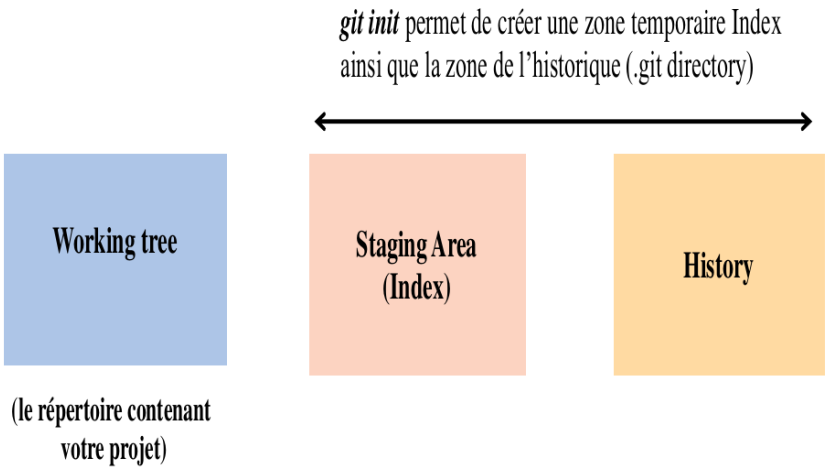
   Le nom et l'email de l'utilisateur sont importants pour savoir quand les changements ont été faits dans ce projet, et qui les a faits. Ainsi, à chaque *commit*, Git enregistre votre nom, votre email et la date de ce *commit*.

   Remarque : pour utiliser d'autres informations sur un dépôt particulier, utilisez **`git config --local user.name "votre nom"`** dans ce dépôt.

   Remarque : sans la troisième commande, **`git init`** nomme la branche principale *master*. GitHub et GitLab utilisent *main* par défaut depuis 2020 ; nous utiliserons donc *main* dans cet atelier.

2. **Initialisation :** créez un répertoire *TpDepot*, initialisez-y un nouveau dépôt Git avec **`git init`**, et créez un fichier nommé Hello (.java ou .txt, ...) affichant un simple message.

   (Notons que la commande **`git init`** crée un sous-répertoire caché `.git` dans notre répertoire *TpDepot*. C'est lui qui contiendra la zone de transit (*staging area*, ou index) et l'historique des révisions.)



   Rappel :

   Pour être intégrée dans l'historique des révisions du dépôt (pour être « commitée »), chaque modification doit passer par les étapes suivantes :

   (a) la modification est d'abord effectuée dans le répertoire du projet (le *working tree*) ;

   (b) elle est ensuite placée dans une zone temporaire (*staging area*) nommée index, avec la commande **`git add nom_fichier`** ;

   (c) enfin, ce qui a été placé dans l'index est enregistré dans l'historique des révisions avec la commande **`git commit -m "message décrivant la modification"`**.

3. **Premier commit :**

   (a) Vérifiez avec la commande **`git status`** l'état dans lequel se trouve votre dépôt. La commande devrait mentionner des fichiers non suivis (*untracked files*). Jusqu'ici, les modifications sont présentes uniquement dans le *working tree*.

   (b) Préparez votre fichier *Hello.txt* (ou un fichier avec n'importe quelle extension) pour un commit, en l'ajoutant à l'index avec la commande **`git add Hello.txt`**.

   (c) Utilisez à nouveau **`git status`** pour vérifier que vos modifications ont été placées dans l'index. Puis, utilisez **`git diff --staged`** (synonyme : **`git diff --cached`**) pour observer les différences entre l'index et la dernière version présente dans l'historique (ici, tout le fichier apparaît comme nouveau).

   (d) Enregistrez maintenant vos modifications dans le dépôt avec la commande **`git commit -m "message lié à ce commit"`**.

   (e) Réexécutez la commande **`git status`**.

   (f) Affichez l'historique des changements effectués dans le dépôt avec la commande **`git log`**. Notons que chaque commit possède un *hash* unique.

4. **Deuxième commit :**

   (a) Modifiez votre fichier Hello.txt, et ajoutez un deuxième fichier readme.txt.

(b) Vérifiez l'état des modifications avec **`git status`**. Utilisez également la commande **`git diff`** pour voir les différences entre un fichier suivi (*tracked file*) dans le *working tree* et l'index. Pourquoi readme.txt n'apparaît-il pas ?

(c) Exécutez la commande **`git add .`** pour ajouter à l'index toutes les modifications effectuées dans le répertoire.

(d) Exécutez à nouveau **`git status`**, puis **`git diff --staged`**.

(e) Validez vos modifications avec **`git commit -m "votre message"`**.

(f) Réexécutez **`git status`**.

(g) Regardez à nouveau l'historique des modifications avec **`git log`**. Qu'est-ce que vous remarquez ?

II. Annuler des changements

Les commandes `git restore` et `git switch` existent depuis Git 2.23 (2019). Les anciennes commandes équivalentes sont indiquées entre parenthèses ; elles fonctionnent toujours.

1. **Annuler une modification du working tree :** modifiez Hello.txt sans faire de **`git add`**. Observez **`git status`** et **`git diff`**, puis annulez la modification avec **`git restore Hello.txt`** (ancienne forme : **`git checkout -- Hello.txt`**). Le fichier reprend la version présente dans l'index.

2. **Retirer une modification de l'index :** modifiez à nouveau Hello.txt, puis exécutez **`git add Hello.txt`**. Retirez-la de l'index avec **`git restore --staged Hello.txt`** (ancienne forme : **`git reset HEAD Hello.txt`**). Vérifiez avec **`git status`** : l'index reprend la version du dernier commit, mais votre modification est **toujours présente** dans le working tree. Annulez-la ensuite comme à la question précédente.

3. **Récupérer un fichier supprimé :**

   (a) Supprimez le fichier readme.txt avec **`git rm readme.txt`**, puis validez avec **`git commit -m "suppression de readme"`**.

   (b) Affichez l'historique de ce fichier : **`git log --oneline -- readme.txt`**. Le commit le plus récent est celui de la suppression : le fichier n'y existe plus.

   (c) Récupérez la version du commit **précédant** la suppression (notez le `^` après le hash) :
   `git restore --source <hash_suppression>^ readme.txt`
   (ancienne forme : **`git checkout <hash_suppression>^ -- readme.txt`**)

   (d) Vérifiez l'état du fichier, puis ajoutez-le et validez-le de nouveau.

4. **Annuler un commit :** la commande **`git revert <hash>`** crée un **nouveau commit** qui défait les changements introduits par le commit indiqué, sans réécrire l'historique. Testez-la sur le commit de suppression de readme.txt (sur une copie de votre dépôt ou avant la question précédente), puis observez **`git log`**.

III. Les branches Git

1. Définissez un alias pour afficher le graphe de l'historique des commits :
   `git config --global alias.graph "log --all --decorate --oneline --graph"`
   Vous pourrez ensuite simplement utiliser **`git graph`**.

2. À partir de la branche principale de développement (*main*), créez deux nouvelles branches nommées *develop* et *bug-fix* :
   `git branch develop`
   `git branch bug-fix`
   Listez les branches avec **`git branch`** : l'astérisque indique la branche courante.

3. Basculez sur la branche *develop* :
   `git switch develop` (ancienne forme : **`git checkout develop`**)

4. Modifiez la **première ligne** de Hello.txt, puis validez tous les changements effectués :
   `git commit -am "Modification sur develop"`
   (l'option `-a` ajoute automatiquement à l'index les fichiers déjà suivis).

5. Affichez le graphe de l'historique des commits.

6. Basculez sur la branche *bug-fix*. Que contient maintenant Hello.txt ? Pourquoi ?

7. Sur la branche *bug-fix*, modifiez **la même première ligne** de Hello.txt, mais différemment, puis validez :
   `git commit -am "Modification sur bug-fix"`

8. Affichez le graphe de l'historique des commits : les deux branches ont divergé.

9. **Fusion fast-forward :** placez-vous sur la branche *main*, puis fusionnez la branche *develop* dans *main* :
   `git switch main`
   `git merge develop`
   Git affiche « Fast-forward ». Affichez le graphe : pourquoi aucun nouveau commit n'a-t-il été créé ?

10. **Fusion avec conflit :** fusionnez maintenant la branche *bug-fix* dans *main* :
    `git merge bug-fix`
    Git signale un conflit sur Hello.txt.

    (a) Exécutez **`git status`** : Hello.txt apparaît dans la rubrique *Unmerged paths*.

    (b) Ouvrez Hello.txt et observez les marqueurs de conflit :
    `<<<<<<< HEAD` (version de la branche courante, *main*), `=======` (séparateur), `>>>>>>> bug-fix` (version de la branche fusionnée).

    (c) Choisissez le contenu final (l'une des versions, ou une combinaison des deux), et supprimez les trois lignes de marqueurs.

    (d) Marquez le conflit comme résolu avec **`git add Hello.txt`**, puis finalisez la fusion avec **`git commit`**.

    (e) *Remarque :* à tout moment avant le commit, **`git merge --abort`** annule la fusion et ramène le dépôt à son état précédent.

11. Affichez le graphe : observez le commit de fusion et ses deux parents.

12. Supprimez les branches fusionnées : **`git branch -d develop bug-fix`**

IV. Dépôt distant GitHub

1. Pousser les modifications sur votre dépôt distant

1. Créez un compte GitHub.

2. Créez un nouveau dépôt (*repository*) vide sur GitHub, sans README.

3. **Authentification :** GitHub n'accepte plus le mot de passe du compte pour les commandes Git en HTTPS. Créez un jeton d'accès personnel (*Settings* → *Developer settings* → *Personal access tokens*) à utiliser à la place du mot de passe, ou configurez une clé SSH.

4. Pour ajouter votre dépôt distant, exécutez la commande **`git remote add`** dans le répertoire contenant votre dépôt local :
   `git remote add origin https://github.com/user/repo-name.git`
   (*origin* est le nom conventionnel du dépôt distant principal.)

5. Vous pouvez vérifier les liens distants enregistrés avec : **`git remote -v`**

6. Pour envoyer la version actuelle de votre historique sur le dépôt distant, utilisez :
   `git push -u origin main`
   (Si votre branche s'appelle encore *master*, renommez-la d'abord avec **`git branch -M main`**.)

2. Manipulation d'un dépôt existant

Il s'agit du cas le plus simple, où un dépôt distant existe déjà.

- Récupérez une copie de ce projet dans votre répertoire personnel au moyen de la commande :
  `git clone https://github.com/username/repo-name.git`
- Ajoutez un fichier avec **`git add`**.
- Validez vos modifications avec **`git commit`**.
- Envoyez vos changements sur GitHub avec **`git push`**.
- Récupérez les changements faits par d'autres avec **`git pull`** (qui équivaut à **`git fetch`** suivi de **`git merge`**).

3. Travail en binôme

- L'étudiant A ajoute l'étudiant B comme collaborateur de son dépôt (*Settings* → *Collaborators*) ; B clone le dépôt.
- A et B modifient chacun la même ligne d'un même fichier, puis valident localement.
- A pousse en premier. Lorsque B pousse à son tour, son **`git push`** est refusé : pourquoi ?
- B exécute **`git pull`**, résout le conflit comme dans la partie III, valide, puis pousse.